

Azure Service Bus – Defer and Receive a Deferred Message

1. What is Message Defer?

Defer means keeping a message aside temporarily because the receiver cannot process it at that moment.

The receiver plans to process the message at a more appropriate time in the future.

Simple Example

```
Message received
  ↓
Cannot process now
  ↓
Defer message
  ↓
Message kept aside
  ↓
Process it later
```

2. What Happens When a Message Is Deferred?

When the message is deferred:

- The message **remains in the queue**.
- Its **delivery count increases by 1**.
- Its state changes from **Active** → **Deferred**.
- It is not normally returned by a regular receive operation.
- The application needs the message's **sequence number** to retrieve it later.

Example

Before defer:

```
Message:
"This message will be deferred"
```

```
Delivery Count: 0  
State: Active
```

After defer:

```
Message:  
"This message will be deferred"  
  
Delivery Count: 1  
State: Deferred
```

3. Defer a Message Using C

After receiving the message:

```
ServiceBusReceivedMessage message =  
    await receiver.ReceiveMessageAsync();
```

We can defer it using:

```
await receiver.DeferMessageAsync(message);
```

Important Code

```
await receiver.DeferMessageAsync(message);
```

This changes the message's state to **Deferred**.

4. Why Do We Need a Sequence Number?

After a message is deferred, we need a way to identify that exact message when we want to process it later.

Azure Service Bus provides a **Sequence Number** for each message.

Example:

```
Sequence Number = 16
```

We can use this number to retrieve the deferred message.

Important Point

Sequence Number = Identifier used to retrieve a specific deferred message.

5. Receiving a Deferred Message

To retrieve the deferred message, use:

```
ServiceBusReceivedMessage deferredMessage =  
    await receiver.ReceiveDeferredMessageAsync(16);
```

Here:

```
16  
↓  
Sequence Number
```

The result is stored in:

```
deferredMessage
```

Now we can access the original message again.

6. Complete Practical Flow

```
1. Receive Message  
  ↓  
2. Cannot Process Now  
  ↓  
3. Defer Message  
  ↓  
4. Message State = Deferred  
  ↓
```

```
5. Delivery Count increases
   ↓
6. Store/remember Sequence Number
   ↓
7. Later retrieve using Sequence Number
   ↓
8. Process Message
   ↓
9. Complete Message
```

7. Practical Example

Suppose the queue contains:

```
"This message will be deferred"
```

Initially:

```
Delivery Count = 0
State = Active
```

The application receives it:

```
var message =
    await receiver.ReceiveMessageAsync();
```

But the application cannot process it yet.

So it defers the message:

```
await receiver.DeferMessageAsync(message);
```

After defer:

```
Delivery Count = 1
State = Deferred
```

Suppose the message's sequence number is:

16

Later, when the application is ready to process it:

```
var deferredMessage =  
    await receiver.ReceiveDeferredMessageAsync(16);
```

Now the application has the original message again.

8. Important Difference: Receive vs Receive Deferred Message

Normal Receive

```
await receiver.ReceiveMessageAsync();
```

Used to receive an available message from the queue.

Receive Deferred Message

```
await receiver.ReceiveDeferredMessageAsync(sequenceNumber);
```

Used to retrieve a **specific deferred message**.

The second method requires the **sequence number**.

9. Azure Portal Verification

Before deferring:

```
Message: This message will be deferred  
  
Delivery Count: 0  
State: Active
```

After executing:

```
await receiver.DeferMessageAsync(message);
```

Refresh the Service Bus Explorer.

You will see:

Message: This message will be deferred

Delivery Count: 1

State: Deferred

The message still exists, but its state has changed.

10. Retrieving the Deferred Message

Suppose the sequence number is:

16

Use:

```
var deferredMessage =  
    await receiver.ReceiveDeferredMessageAsync(16);
```

The message is retrieved:

Deferred Message

↓

Sequence Number = 16

↓

ReceiveDeferredMessageAsync()

↓

deferredMessage

Now we can access its body:

```
Console.WriteLine(  
    deferredMessage.Body.ToString());
```

Output:

```
This message will be deferred
```

11. Real-Time Scenario

Consider an **Order Processing System**.

An order has two steps:

```
Task 1 → Validate Customer  
Task 2 → Process Order
```

The application receives the Task 2 message before Task 1 is completed.

It cannot process Task 2 yet.

Instead of repeatedly trying to process it, the application can defer the message.

```
Task 2 Message  
  ↓  
Task 1 not completed  
  ↓  
Defer Task 2 message  
  ↓  
Task 1 completed  
  ↓  
Retrieve deferred Task 2 message  
  ↓  
Process Task 2
```

This is a good scenario for **message deferral**.

12. Important Interview Questions

Q1. What is message deferral in Azure Service Bus?

Answer:

Message deferral allows a receiver to temporarily keep a message aside when it cannot process the message at that time. The message can be retrieved later using its sequence number.

Q2. What happens to the message state after deferring?

Answer:

The message changes from **Active** to **Deferred**.

Q3. Does a deferred message get deleted?

Answer:

No. The message remains in the queue in a deferred state.

Q4. How do you retrieve a deferred message?

Answer:

Use the message's sequence number with:

```
await receiver.ReceiveDeferredMessageAsync(sequenceNumber);
```

Q5. Why is the sequence number important?

Answer:

The sequence number uniquely identifies the message that we want to retrieve from the deferred messages.

13. Defer vs Abandon

Feature	Abandon	Defer
Message remains in queue	Yes	Yes
Delivery count increases	Yes	Yes
Message available for normal receive	Yes	No
State	Active	Deferred
How to retrieve later	Normal receive	Sequence number
Main purpose	Try processing again	Intentionally process later

Easy Memory Trick

Abandon → Try Again

Defer → Process Later

14. Complete Example

```
using Azure.Messaging.ServiceBus;

class Program
{
    static async Task Main(string[] args)
    {
        string connectionString = "YOUR_CONNECTION_STRING";
        string queueName = "YOUR_QUEUE_NAME";

        ServiceBusClient client =
            new ServiceBusClient(connectionString);

        ServiceBusReceiver receiver =
            client.CreateReceiver(queueName);

        // Receive message
        ServiceBusReceivedMessage message =
            await receiver.ReceiveMessageAsync();

        // Defer message
        await receiver.DeferMessageAsync(message);
    }
}
```

```
// Later, retrieve using sequence number
long sequenceNumber = message.SequenceNumber;

ServiceBusReceivedMessage deferredMessage =
    await receiver.ReceiveDeferredMessageAsync(
        sequenceNumber);

Console.WriteLine(
    deferredMessage.Body.ToString());
    }
}
```

15. Quick Revision

DEFER
↓
Keep message aside temporarily
↓
Message remains in queue
↓
Delivery Count increases
↓
State becomes Deferred
↓
Use Sequence Number
↓
ReceiveDeferredMessageAsync()
↓
Retrieve message later

Most Important Code

Defer

```
await receiver.DeferMessageAsync(message);
```

Receive Deferred Message

```
var message =  
    await receiver.ReceiveDeferredMessageAsync(sequenceNumber);
```

Remember

Defer = Keep the message aside now and retrieve that specific message later using its sequence number.